Reviewer Pack — Minimal Rank of Delone Set Complexity vs Permutation Circuit…

Entry 3117bfb0b88b · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-25 08:29:37 UTC

Minimal Rank of Delone Set Complexity vs Permutation Circuit Depth

Entry ID: `3117bfb0b88b` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-25 08:29:37 UTC

1. Conjecture

Field A (mathematical branch): Tiling Theory (Delone Sets) **Field B** (complexity object): Complexity Theory: Permutation Circuit Depth

Statement:

> ['The minimal rank of the complexity matrix associated with a Delone set is lower bounded by the depth of any permutation circuit computing its characteristic polynomial.', 'For every Delone set D and its characteristic polynomial χ_D, there exists a permutation circuit C such that χ_D = det(C) and the depth of C is at least ρ(D), where ρ(D) is the minimal rank of the complexity matrix of D.', 'If there were a Delone set with a complexity matrix having minimal rank ρ(D) and no corresponding permutation circuit of depth less than ρ(D), then this would be a counterexample.']

Rationale (proposer's reasoning):

> ['Delone sets are geometric objects that have been studied in tiling theory, and their complexity matrices capture the combinatorial information of these tilings.', 'Permutation circuits provide a computational model to study the complexity of functions over permutations, which is relevant for problems like sorting and group isomorphism.', 'This conjecture aims to bridge these two fields by relating the geometric structure of Delone sets to the computational complexity of permutation functions.']

Taxonomy category: `GCT_DET_PERM` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `1464139b51aabab9`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> The conjecture is supported if the mean minimal rank of Delone sets' complexity matrices is greater than or equal to the mean minimum permutation circuit depth required to compute their characteristic polynomials for at least 80% of the seeds, with no seed having a permutation circuit depth less than the corresponding minimal rank.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM_1	LLM_2
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - ('Delone set' AND 'permutation circuit depth') - ('characteristic polynomial' AND 'minimal rank' AND 'tiling theory') - ('complexity matrix' AND 'depth of permutation circuits' AND 'tiling structures')

Top relevant hits considered: - [http://arxiv.org/abs/1902.05441v2] A Note on Entropy of Delone Sets - [http://arxiv.org/abs/1405.4233v3] Ergodicity and dynamical localization for Delone-Anderson operators - [http://arxiv.org/abs/1912.05379v3] Chaotic Delone sets - [http://arxiv.org/abs/1208.5345v2] Generating All Minimal Edge Dominating Sets with Incremental-Polynomial Delay - [http://arxiv.org/abs/1401.4438v1] Integral closure of rings of integer-valued polynomials on algebras - [http://arxiv.org/abs/1412.5071v3] Probabilistic analysis of Wiedemann's algorithm for minimal polynomial computation - [http://arxiv.org/abs/2411.06569v2] Randomized Black-Box PIT for Small Depth +-Regular Non-commutative Circuits - [http://arxiv.org/abs/0911.3482v5] Complexity of Networks (reprise)

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=124, elapsed=240.1s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def gcd(a, b):
    while b:
        a, b = b, a % b
    return a

def lcm(a, b):
    return abs(a * b) // gcd(a, b)

def matrix_multiply(A, B):
    m, n = len(A), len(B[0])
    p = len(B)
    C = [[0] * n for _ in range(m)]
    for i in range(m):
        for j in range(n):
            for k in range(p):
                C[i][j] += A[i][k] * B[k][j]
```

```
    return C

def matrix_determinant(A):
    if len(A) == 1:
        return A[0][0]
    det = Fraction(0)
    sign = Fraction(1, 1)
    for j in range(len(A)):
        submatrix = [row[:j] + row[j+1:] for row in A[1:]]
        det += sign * A[0][j] * matrix_determinant(submatrix)
        sign *= -Fraction(1, 1)
    return det

def char_poly(D):
    n = len(D)
    A = [[0] * (n + 1) for _ in range(n + 1)]
    for i in range(n):
        for j in range(n):
            A[i][j] = D[i][j]
    A[n][n-1] = -sum(A[i][i] for i in range(n))
    A[n][n] = 1
    det_A = matrix_determinant(A)
    return [det_A]

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = random.randint(5, 40)
    D = [[random.randint(-10, 10) for _ in range(n)] for _ in range(n)]
    det_D = matrix_determinant(D)
    char_poly_D = char_poly(D)
    min_rank = len(D)
    min_depth = float('inf')

    for i in range(100):
        C = [[random.randint(-10, 10) for _ in range(n)] for _ in range(n)]
        det_C = matrix_determinant(C)
        if det_C == det_D:
            depth = 0
            while True:
                depth += 1
                C_new = []
                for j in range(n):
                    row = [C[i][j] for i in range(n) if i != j]
                    C_new.append(row)
                det_C = matrix_determinant(C_new)
                if det_C == det_D:
                    break
            min_depth = min(min_depth, depth)

    return {
        "metric_name": "min_rank",
        "metric_value": min_rank,
        "instances_tested": 100,
        "conjecture_holds": min_rank <= min_depth,
        "counterexample": "" if min_rank <= min_depth else f"Det(D)={det_D}, Det(C)={det_C}"
    }
```

```python
if __name__ == "__main__":
    import sys
    seeds = [int(x) for x in sys.argv[1:]] or list(range(2, 30))

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
    std_metric_value = math.sqrt(sum((r["metric_value"] - mean_metric_value) ** 2 for r in results
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"Det(D) != Det(C)\" first_failing_seed={first_fa
    else:
        print("RESULT: INCONCLUSIVE no seeds supported the conjecture")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
TIMEOUT after 240s
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test timed out before producing data, making it impossible to verify the conjecture's support conditions. | next: NONE

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	10753
2	preregistration	ollama_remote	glm4:latest	0	0	5588
3	novelty	ollama_remote	glm4:latest	0	0	4832
4	novelty	ollama_remote	glm4:latest	0	0	6434

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	39371
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10894
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9480
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11167
9	judge	ollama_remote	glm4:latest	0	0	8412

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 106930 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/3117bfb0b88b.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/3117bfb0b88b.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/3117bfb0b88b.tar.gz` (if generated)